yarg-song-server

A self-hosted song library server for YARG (Yet Another Rhythm Game).

Point it at a folder of songs, run it on whatever is always-on in your house, and every YARG machine on the network plays from the same library. It is a single static binary — Docker, Raspberry Pi, macOS and Windows all run the same code.

> **Status: the server runs and serves songs. There is no sync client yet.**
>
> It scans a library, serves a browsable and searchable catalog over HTTP, answers "which of these hashes am I missing?" in bulk, and hands out any song as a `.sng` — packing a loose folder on demand. What is still missing is the small companion that pulls a library into a local songs folder, which is what makes it useful without touching the client. See `docs/ROADMAP.md` and `docs/API.md`.
>
> What "works" means here is measured, not asserted. Archives this tool writes are decoded by the reference `SngCli`; **a real YARG install scanned 22 archives served by the running server and accepted 20**, the two refusals being corpus cases built to be refused and both independently flagged by our own scanner. Details in `docs/TEST-CORPUS.md`.

Design in one paragraph

The server is a **content source, not a game modification**. It serves ordinary `.sng` files that an unmodified YARG already knows how to read, so the first useful version needs no client changes at all — a small sync companion pulls the library into a normal songs folder. Native in-client browsing comes later, and separately, as an upstream conversation.

Songs are identified by `SHA1(chart file bytes)`, which is exactly how YARG itself identifies them. Client and server therefore agree precisely on "do I already have this song", with no heuristics and no fuzzy matching.

What it will and will not do

Will

- Ingest loose song folders (`song.ini` + `notes.mid` / `notes.chart` + stems), `.sng` containers, and zipped versions of either
- Serve a browsable, searchable catalog over HTTP, sorted by the same attributes YARG sorts by
- Serve songs as `.sng`, and answer "which of these hashes am I missing?" in bulk
- Run on `linux/amd64`, `linux/arm64` (Raspberry Pi), macOS and Windows

Will not, ever

- Decrypt Rock Band CON packages or encrypted moggs. Upstream lists CON decryption as out of scope and will reject PRs for it on sight; it also carries real legal exposure. `.con`, `_rb3con` and `.pkg` are refused on ingest with a clear message.
- Generate YARG's `songcache.bin`. Not because it is unreadable — it is a plain binary file — but because it stores **absolute local paths**, so a cache built here is meaningless on your machine; because its version is a date stamp checked with no compatibility window and no migration path, and its field layout is internal with no version of its own; and because getting any of it wrong fails *silently*, with YARG quietly rebuilding the cache while the tool appears to have worked.
- Distribute copyrighted audio. Charts and audio are separable by design.

Documentation

Document	What it covers
`docs/ROADMAP.md`	Phases, build order, exit criteria, blockers
`docs/API.md`	Every HTTP endpoint, what it promises, and what it explicitly does not claim
`docs/ADR-001-server-architecture.md`	Why Go, why two repos, why sync-first, why LGPL
`docs/ADR-002-v1-store.md`	Why the catalog lives in memory, why packed archives are cached, and the two places this server deliberately sorts differently from the client
`docs/SYNC-CLIENT.md`	`yarg-sync`: flags, what it refuses to touch, why files are named by chart hash, and the Windows Defender false positive
`docs/DEPLOY-VAULT2.md`	Running the image as a container — the `chown 65532` and pool-path details that matter, registry auth, and the first end-to-end result off the dev machine

Document	What it covers
`docs/SOURCES.md`	What is already documented and where — **read this before reverse-engineering anything**
`docs/TEST-CORPUS.md`	Where test input comes from, and what a real YARG install said about it
`docs/research/yarg-song-formats.md`	The `.sng` binary layout, `song.ini` keys, the metadata model, song identity, and a difficulty assessment for every part of a Go reimplementation

Read the research document before writing any parser code. It is the reason the scope is what it is.

Running it

```
make build
./bin/yarg-song-server --songs /path/to/songs --data ./data --listen :8080
```

Then `GET /api/v1/library` to see what it indexed, `GET /api/v1/songs?q=&sort=artist` to browse, and `GET /song/{chart_hash}.sng` to pull a song. Every endpoint is in `docs/API.md`.

The library is read only ever; `--data` is where the server keeps its own state, including archives packed from loose folders, and must be writable.

It is unauthenticated and read-only. Run it on your LAN, not on the internet.

Syncing a library into stock YARG

`yarg-sync` pulls a server's library into an ordinary songs folder. **YARG is not modified and does not know the server exists** — it sees files in a folder, which is all it has ever needed:

```
./bin/yarg-sync -server http://pi.local:8080 -songs "/path/to/YARG Songs" -dry-run
```

It writes `<chart_hash>.sng` files and nothing else, and never reads, renames or deletes anything it did not write — so it is safe to point at a folder that already holds your own charts. `-prune` is off by default. Every download is verified by re-deriving the song's identity from the bytes that arrived, so a truncated or substituted file fails closed rather than landing.

Full behaviour, including the deterministic answer when two packages share a chart, is in `docs/SYNC-CLIENT.md`.

> **Windows:** an unsigned Go binary that downloads files can trip a machine-learning malware verdict. Defender did exactly that to `yarg-sync` on 2026-09-05 — `Trojan:Win32/Bearfoos.A!ml` — and then stopped on its own within the hour once the classifier was revised upstream. If you hit it, re-test before doing anything drastic, and please don't add an antivirus exclusion or turn protection off to run this. Signing the release binary is the real fix and is on the list. Details and the escalation path are in the document above.

Configuration

Every setting can be a flag or a line in a config file, with the same name for both. A flag beats the file, the file beats the defaults, and `./yarg-song-server.conf` is read automatically if it exists:

```
./bin/yarg-song-server --write-config > yarg-song-server.conf
```

An unknown setting in that file is an error rather than a warning. A typo that is silently ignored leaves you certain you changed something you did not.

One setting is worth knowing about before you deploy: `pack_cache_max`, default 2 GiB. A song stored as a loose folder is packed to `.sng` on first request and the archive is kept, and an archive is within about 2% of the size of the folder it came from — so an unbounded cache over a loose library eventually needs a second copy of that library on the data disk. That is fine on a server and fatal on a Raspberry Pi whose `--data` is the SD card. Hitting the bound costs a re-pack and never data: the archive is rebuilt byte-identically and its package hash, which comes from the content, does not change. Raise it if you have the disk; `0` means unbounded and the server says so loudly at start if you choose it.

Building

```
make build      # host binary into ./bin
make test       # unit tests, with the race detector
make release    # every promised platform into ./dist
make docker     # multi-arch image
```

CI runs the same checks on every push — `gofmt`, `go vet`, the suite with and without `-race`, all six release targets, and an assertion that the Dockerfile's Go is not older than `go.mod` requires. On the default branch it also builds and pushes a **multi-arch container image** for `linux/amd64` and `linux/arm64`. See `.gitlab-ci.yml`.

That image needs no emulation: Go cross-compiles, and the Dockerfile pins its build stage with `FROM --platform=$BUILDPLATFORM`, so the toolchain runs at the host's architecture and Go emits the arm64 binary. If you are ever tempted to install qemu or register `binfmt_misc` to build this project, something else has gone wrong.

Trying the scanner directly

The CLI is still there, and is how the parsers get pointed at a real collection. Point it at a song library and it prints the catalog as JSON, one song per record:

```
./bin/yarg-song-server scan /path/to/songs
```

It recognises loose song folders and `.sng` archives, walks nested directories, skips folders that hold no chart, and reports an unreadable song on stderr rather than abandoning the scan.

Repacking a loose folder into a `.sng` also works:

```
./bin/yarg-song-server pack /path/to/song-folder out.sng
```

The chart is copied byte for byte, so the song's identity is unchanged. Archives written this way have been decoded by the reference `SngCli` and scanned by a real YARG install — see `docs/TEST-CORPUS.md`.

Licence

LGPL-3.0-or-later, matching YARG and YARG.Core. See `LICENSE`.

This project is not affiliated with or endorsed by YARC. It stands with upstream against piracy: it ships no content, and it will not help you play content you do not own.